

int (all types)
short
char

Range of the data type

Minimum and maximum
How long

Formula

$$2^{n-1}$$

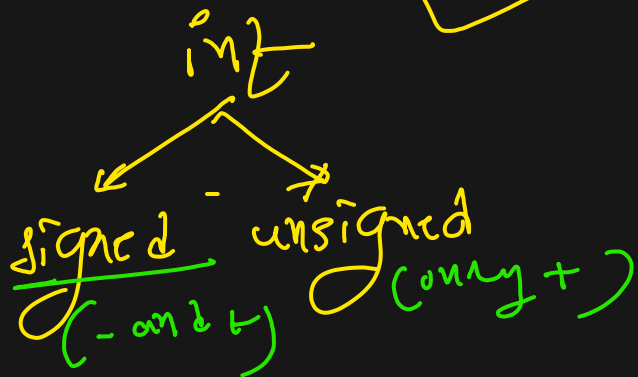
n = size of the data type in bits ✓

int = 4 bytes = 32 bits

$$\begin{aligned} & \therefore 2^{n-1} \\ & = 2^{32-1} \\ & = 2^{31} \end{aligned}$$

Short int = 2 bytes = 16 bits

unsigned 0 to 2^{15}
0 to $(32768) + 2$
= 0 to 65,535
(min -1)



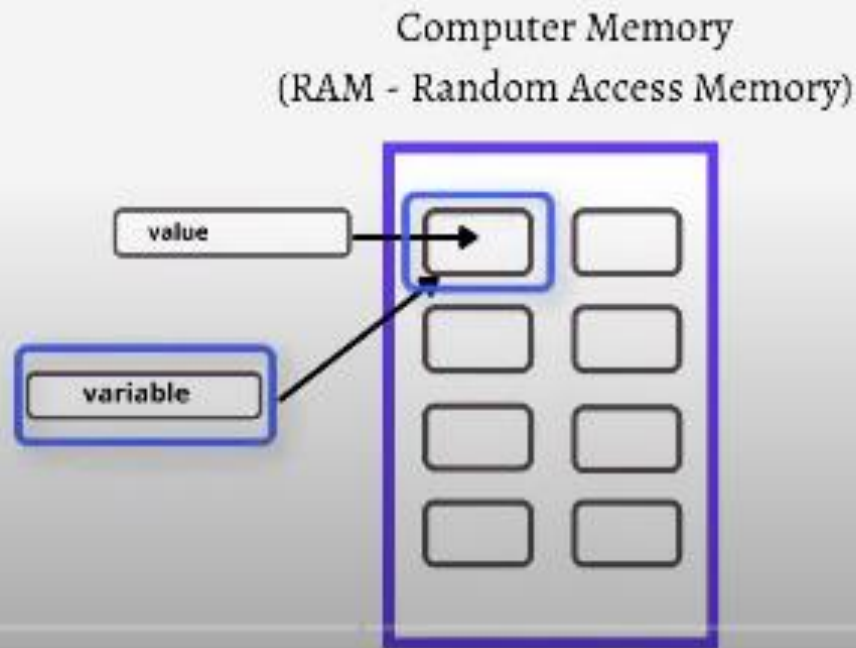
range { -2,147,483,648 to 2,147,483,647 }

unsigned

✓ 0 to $(2,147,483,648) + 2$
= 0 to 4,294,967,295 ✓

Variable

- Variable is a name that use to allocate computer memory to store a value
- We can access the stored value using of the variable name
- The allocated memory size depends on **Data Type** of the variable



Variable Declaration

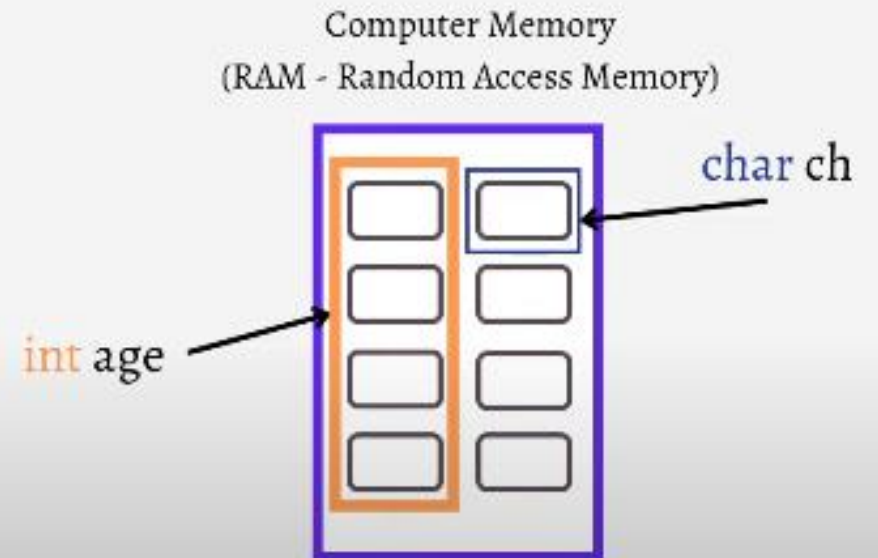
Variable Declaration

`<data_type> <variable_name>;`

Example:

```
int age;
```

```
char ch;
```



Rules for defining a Variable

- A variable name can only have alphanumeric characters (a-z , A-Z , 0-9) and underscore(_). Doesn't allow any other special characters like @, \$ etc.

E.g. `int first$name;`
`int num&;`



`int first_name;`
`int age;`
`int _temp;`



- The first character of a variable name can only contain alphabet (a-z, A-Z) or underscore (_). We can't use any digit as a first character

E.g. `2num`



`int num2`



- C Keywords are not allowed to use as a variable name
- C is case-sensitive programming language i.e. Age and age are two different variable in C.

E.g. `int int`



`int INT`



- Blank spaces are not allowed in a variable name.

E.g. `int first name;`



`int first_name;`



DATA TYPE





BOOK

Id	Name	Quantity	Price	Format
1001	C Programming	100	\$5.20	H
1002	Java Programming	105	\$34.50	P
1003	C++ Programming	20	\$20.20	H
1004	Python for Beginners	100	\$36.50	P

H = Hardcover Book

P = Paperback Book

What is Data Type

- Data Type defines the **type of data** being used in the program
- According to the Data Type, the **memory allocated** in the computer memory



Example

```
int bookId = 1001;  
char name[] = "C Programming";  
int quantity = 100;  
float price = 25.20;  
char format = 'H';
```

Book



Group of Data Types

1. Primary Data Types

◦ int, char, float, double, void

2. Derived Data Types

◦ array, structure, union, pointer

3. User-Define Data Types

◦ enum, typedef

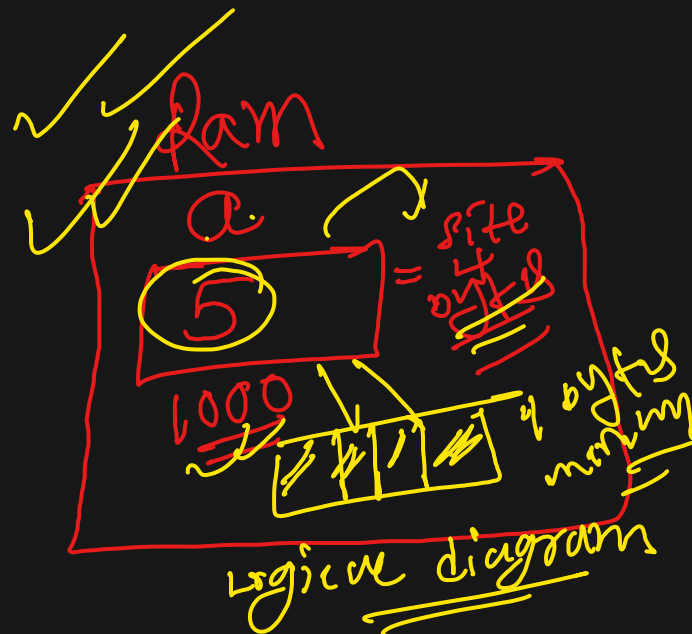
Dynamic Type

int a

018

int a = 5 integer value

int = 4 bytes
float = 4 bytes
double = 8 bytes
char = 1 byte



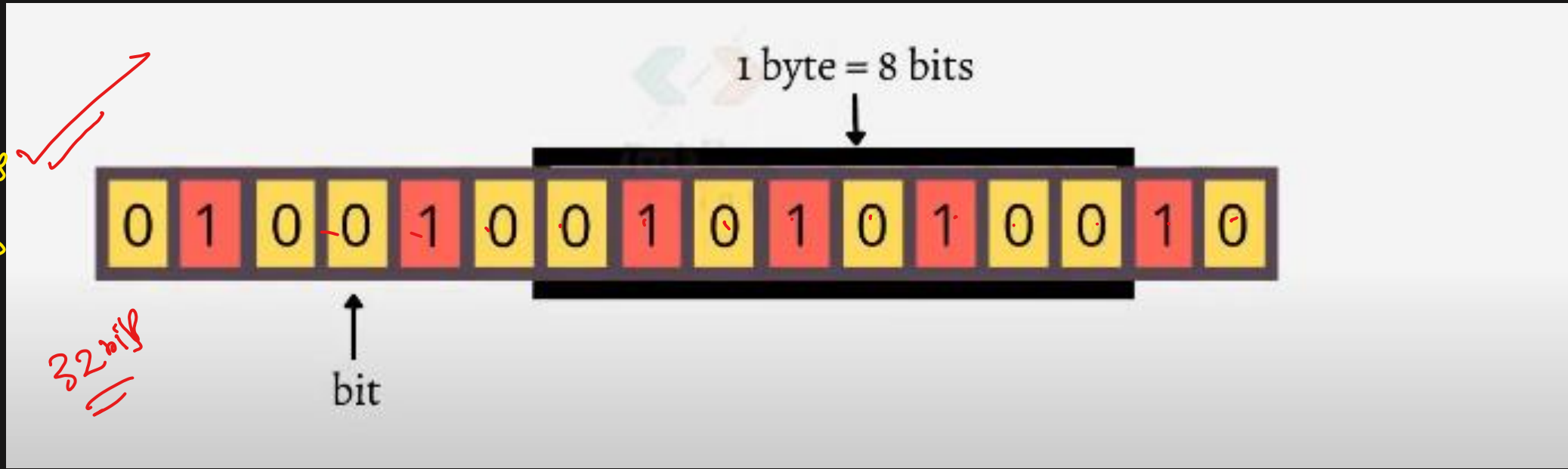
① $a = 5$ (no)
→ decimal
= 0101

How data is stored in computer memory

(bits)

Computer stores information in the form of bytes (1s and 0s) in all its memory.
Binary

② int
= 4 bytes
↓
bits
1 byte = 8 bits
4 bytes = 4 x 8
= 32 bits



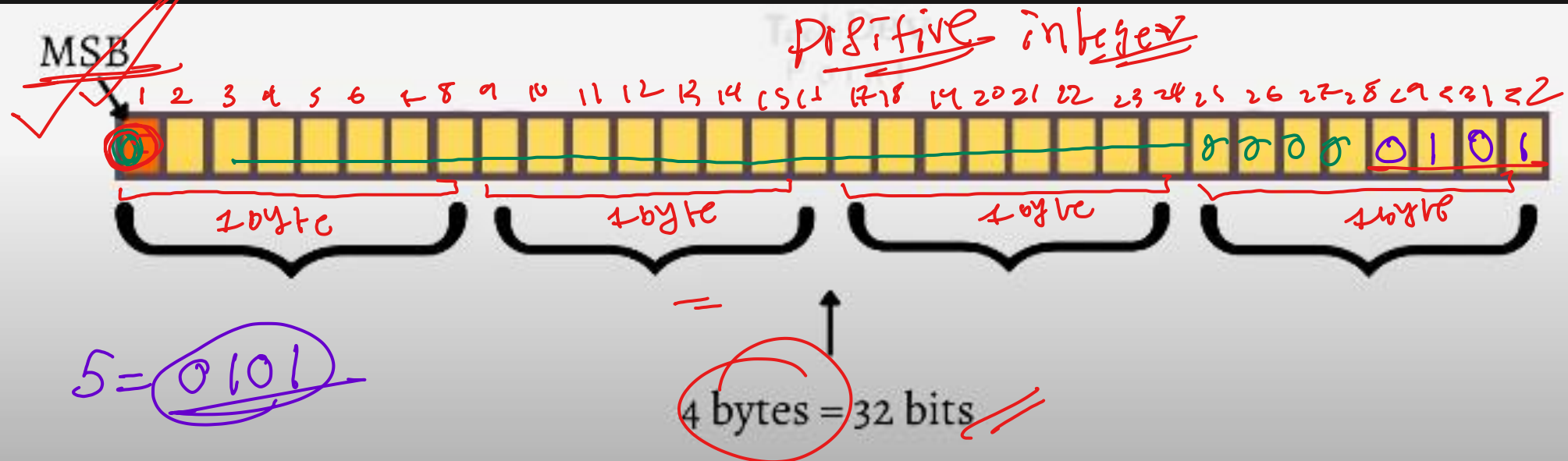
double = 8 bytes = 8×8 bits = 64 bits
char = 1 byte = 1×8 = 8 bits

How integers are stored in memory

MSB (most significant bit) = indicate whether the number is positive or negative.

MSB = 0, for positive numbers

MSB = 1, for negative numbers



How **positive integers** are stored in memory

`int var = 36;`

Binary equivalent of 36 is $(100100)_2$

$36/2 = 18$	remainder	0
$18/2 = 9$	remainder	0
$9/2 = 4$	remainder	1
$4/2 = 2$	remainder	0
$2/2 = 1$	remainder	0
$1/2 = 0$	remainder	1
$0/2 = 0$	remainder	0

MSB

var

0 1 0 0 1 0 0

4 bytes = 32 bits

8 4 2 1

How **negative integers** are stored in memory
negative integers are stored as **two's complement**

Steps to find two's complement:

✓ **Step 1:** Convert decimal to binary

Step 2: **1's complement** - inverting binary bits (i.e. switching 0 to 1 and 1 to 0)

Step 3: **2's complement** - just add 1 to 1's complement

Binary addition

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 10 \end{array}$$

Example

signed int var = -36;

Step 1: Binary equivalent of 36 is $(100100)_2$

MSB



Step 2: 1's complement - inverting binary bits (i.e. switching 0 to 1 and 1 to 0)

1's complement of 36



Step 3: 2's complement - just add 1 to 1's complement

MSB = 1 indicates negative number



int a = 5

Binary add

+1

-36

MSB = 1 indicates negative number

1 0 1 1 1 0 0

2's complement

```
int main()
{
    signed int var = -36;
    printf("%d", var);
    return 0;
}
```

%d
prints Signed Integer

0 1 0 0 1 0 0

convert binary to decimal

-36

prints the decimal results by adding - (minus) sign

Handwritten red notes and scribbles on the left side of the image, including a large scribble at the top, a box, and various arrows and symbols.